

Product Requirement Document (PRD): SMS Notification Microservice

1. Document Overview

1.1 Purpose

This document outlines the product and technical requirements for building a standalone **SMS Notification Microservice** using NestJS. The goal is to provide a unified, highly available, resilient, and extensible service that manages sending SMS messages across multiple third-party providers.

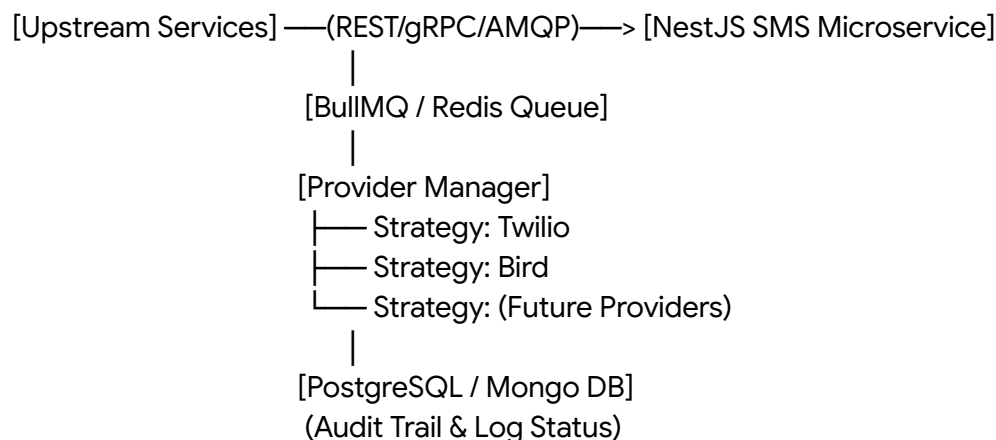
1.2 Target Audience

- **Primary Implementer:** Junior NestJS Developer
- **Stakeholders:** System Architect, Backend Team, QA Engineer

1.3 Key Objectives

1. **Abstraction Layer:** Shield upstream services from third-party vendor implementation details.
2. **Multi-Provider Support:** Initial launch supporting **Twilio** and **Bird** (formerly MessageBird), designed so adding a 3rd or 4th provider takes minimal code changes.
3. **Zero Message Loss (High Reliability):** Implement robust queuing, retries, dead-letter processing, and automatic provider failover.
4. **Idempotency:** Prevent duplicate SMS delivery to end-users caused by retries or network blips.

2. High-Level Architecture & Workflow



Workflow Steps

1. Upstream services request an SMS dispatch by passing recipient phone number, message payload, and an idempotencyKey.
2. The service validates input, checks idempotency in Redis/DB, logs the request as PENDING, and pushes the job to a durable queue (e.g., BullMQ).
3. Queue worker picks up the job and attempts dispatch via the **Primary Provider** (e.g., Twilio).
4. If primary provider succeeds, job state updates to SENT.
5. If primary provider fails (e.g., rate-limited, outage, API 5xx), the service retries or fails over seamlessly to the **Secondary Provider** (e.g., Bird).
6. Webhook callbacks from Twilio/Bird update the status in the database (e.g., DELIVERED, FAILED).

3. Detailed Requirements

3.1 Functional Requirements

FR-1: Request Ingestion & Validation

- The service must expose an internal endpoint (REST endpoint or Message Broker handler like RabbitMQ/Redis/Kafka).
- Payload validation using class-validator and class-transformer in NestJS.
- **Fields required:**
 - recipientPhone (E.164 format, e.g., +14155552671)
 - messageBody (string, standard text, max characters configurable)
 - idempotencyKey (string, unique client identifier to prevent duplicate sends)
 - metadata (optional JSON object for tracking user IDs, source service, etc.)

FR-2: Multi-Provider Abstraction (Strategy Pattern)

- Implement a provider strategy pattern using standard TypeScript interfaces (e.g., ISmsProvider).
- Launch Providers:
 1. **Twilio** (TwilioProvider)
 2. **Bird** (BirdProvider)
- Adding a new provider must only require implementing ISmsProvider and registering it in a factory/registry without modifying core message orchestration code.

FR-3: Dynamic Provider Selection & Automatic Failover

- The service should maintain a priority order for active providers (e.g., Priority 1: Twilio, Priority 2: Bird).
- If sending via Priority 1 fails due to a network error or 5xx server error from the provider, the system must automatically fallback to Priority 2.

- Provider selection logic must be configurable via environment variables or a configuration service.

FR-4: Delivery Webhook Handling

- Expose public HTTP endpoints for receiving delivery reports (DLR) from supported vendors:
 - POST /webhooks/twilio
 - POST /webhooks/bird
- Webhook controllers must authenticate/verify webhook signatures (using provider secrets) to ensure authenticity.
- Update message delivery status in the database (DELIVERED, UNDELIVERED, REJECTED).

3.2 Non-Functional & Reliability Requirements (CRITICAL)

NFR-1: At-Least-Once Guaranteed Delivery

- Messages **must never be lost** once accepted by the service.
- All incoming requests must be written to a persistent queue (e.g., Redis via BullMQ or RabbitMQ) *before* attempting API calls to providers.

NFR-2: Retries and Exponential Backoff

- Failed provider requests due to transient issues (timeouts, rate limits 429, server errors 500/502/503) must retry automatically.
- Configure exponential backoff algorithm (e.g., retry after 2s, 4s, 8s, 16s).
- Maximum attempt limit: Configurable (default: 3 retry attempts per provider).

NFR-3: Dead Letter Queue (DLQ)

- If a message fails across all retries and all available providers, move the message to a **Dead Letter Queue (DLQ)**.
- Save the full failure state and logs into DB marked as FATAL_FAILURE.
- Provide a management endpoint or CLI tool to re-queue failed messages from DLQ.

NFR-4: Idempotency

- If the service receives duplicate requests with the same idempotencyKey within a configurable timeframe (e.g., 24 hours):
 - If the previous request is processing/sent, return the existing tracking record without re-sending the SMS.
 - Use Redis distributed lock / cache for rapid idempotency checking.

NFR-5: Rate Limiting & Throttling

- Respect each provider's rate limits (e.g., Twilio standard limits or Bird TPS limit).
- Use queue rate-limiting settings (e.g., BullMQ rate limit settings: limiter: { max: 10, duration: 1000 }) to prevent hitting provider 429 limits.

NFR-6: Logging & Observability

- Structured JSON logging using NestJS Winston or Pino logger.
- Log key event milestones: MESSAGE_QUEUED, PROVIDER_ATTEMPT, PROVIDER_FAILOVER, MESSAGE_SENT, WEBHOOK_RECEIVED.
- Sensitive data masking: Phone numbers and message content must be obfuscated in application logs (e.g., +1415***2671).

4. Technical Architecture Guide for Junior Developer

4.1 Recommended Module Structure

Organize the NestJS codebase cleanly using modular design:

```
src/
├── config/           # Environment variables & provider configs
├── modules/
│   ├── sms/         # Main SMS orchestration & API controller
│   │   ├── dto/     # Data Transfer Objects (Validation)
│   │   ├── sms.controller.ts
│   │   └── sms.service.ts
│   ├── queue/       # BullMQ setup & processor
│   │   ├── sms.processor.ts
│   │   └── queue.service.ts
│   ├── providers/   # Provider strategies
│   │   ├── interfaces/
│   │   │   └── sms-provider.interface.ts
│   │   ├── strategies/
│   │   │   ├── twilio.provider.ts
│   │   │   └── bird.provider.ts
│   │   ├── provider.factory.ts
│   │   └── providers.module.ts
│   ├── webhooks/   # Provider Webhook controllers
│   └── database/   # Entity models & repository layer
```

4.2 Strategy Interface Design Tip

Create a strict contract that all SMS providers must fulfill:

// src/modules/providers/interfaces/sms-provider.interface.ts

```
export interface SendSmsOptions {
  to: string;
  body: string;
  referenceId: string; // Internal message ID for tracking
}
```

```
}
```

```
export interface SendSmsResult {  
  success: boolean;  
  providerMessageId?: string;  
  error?: string;  
  isRetryable: boolean; // Indicates if error is transient (e.g., 500, network error)  
}
```

```
export interface ISmsProvider {  
  readonly providerName: string;  
  sendSms(options: SendSmsOptions): Promise<SendSmsResult>;  
}
```

4.3 Database Schema Blueprint

Table/Collection: sms_logs

Field	Type	Description
id	UUID	Primary Key
idempotencyKey	String (Indexed, Unique)	Client idempotency identifier
recipientPhone	String	Recipient phone number (E.164)
messageBody	Text	Sanitized message content
status	Enum	QUEUED, PROCESSING, SENT, DELIVERED, FAILED
attempts	Integer	Total sending attempts across providers
selectedProvider	String	Name of the provider used (twilio, bird)
providerMessageId	String	External ID returned by vendor
lastError	Text	Diagnostic error message if failed

createdAt	Timestamp	Record creation date
updatedAt	Timestamp	Last record update date

5. API & Contract Specifications

5.1 POST /api/v1/sms/send

Request Headers:

Content-Type: application/json

X-Idempotency-Key: 9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d

Request Body:

```
{
  "to": "+14155552671",
  "message": "Your verification code is 482019",
  "metadata": {
    "userId": "usr_123456",
    "purpose": "OTP"
  }
}
```

Response 202 Accepted:

```
{
  "status": "success",
  "data": {
    "messageId": "c8d488e9-f308-43e8-8db9-8df0cb5134ef",
    "status": "QUEUED",
    "createdAt": "2026-08-05T21:30:00.000Z"
  }
}
```

6. Definition of Done (Acceptance Criteria)

- [] NestJS app runs cleanly without warnings or errors.
- [] Endpoints validate inputs correctly (invalid phone numbers return 400 Bad Request).
- [] Unit tests cover ProviderFactory and provider strategy classes (TwilioProvider, BirdProvider).
- [] Integration tests verify that if TwilioProvider throws a retryable exception, BirdProvider automatically executes.
- [] Redis Queue (BullMQ) correctly handles backoff retries on simulated connection

failures.

- [] Duplicate requests using the same X-Idempotency-Key do not trigger duplicate SMS sends.
- [] Webhooks from Twilio and Bird update message status to DELIVERED or FAILED in the database.
- [] Environment variables (TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, BIRD_API_KEY, etc.) are documented in a .env.example file.

7. Next Steps for Developer

1. Review the requirements and setup a new NestJS repository.
2. Install NestJS BullMQ module (@nestjs/bullmq), TypeORM/Prisma, and official vendor SDKs (twilio, @bird/sdk or raw HTTP via @nestjs/axios).
3. Build the core ISmsProvider interface and register mock providers before writing vendor-specific code.
4. Set up local Docker services for PostgreSQL and Redis to test queuing and persistence.